

Institut Teknologi Bandung
Sekolah Teknik Elektro dan Informatika
Teknik Informatika

Laporan Tugas Besar
Milestone 4: Intermediate Code Generator

GTFOTBFO (HBB)

Nathanael Imandatua Manurung - 13524021
Nicholas Wise Saragih Sumbayak - 13524037
Kurt Mikhael Purba - 13524065
Rava Khoman Tuah Saragih - 13524107

Semester II / 2025–2026

Laboratorium Ilmu dan Rekayasa Komputasi

Daftar Isi

1	Ringkasan Pengerjaan Milestone 4	2
1.1	Struktur File Milestone 4	2
2	Implementasi Intermediate Code Generator	3
2.1	Arsitektur Instruksi P-Code	3
2.2	Operasi Aritmatika dan Logika (OPR)	3
2.3	Strategi Pembangkitan Kode	4
3	Implementasi Interpreter dan Stack Machine	5
3.1	Model Memori <i>Stack Machine</i>	5
3.2	Mekanisme Eksekusi <i>Interpreter</i>	5
3.3	Penanganan Kesalahan <i>Runtime</i>	5
4	Pengujian	6
4.1	Kasus 1: Program Dasar (<code>pos_basic.txt</code>)	6
4.2	Kasus 2: Aritmatika (<code>pos_arith.txt</code>)	6
4.3	Kasus 3: If-Else (<code>pos_if_else.txt</code>)	6
4.4	Kasus 4: While Loop (<code>pos_while.txt</code>)	7
4.5	Kasus 5: For Loop (<code>pos_for.txt</code>)	7
4.6	Kasus Tambahan: Prosedur dengan Parameter	7
5	Kesimpulan	9

1 Ringkasan Pengerjaan Milestone 4

Milestone 4 merupakan tahap akhir kompilasi bahasa Pascal sederhana, yaitu pembangkitan kode perantara (intermediate code) dan eksekusinya menggunakan interpreter stack-based. Kelompok kami mengimplementasikan generator kode P-Code untuk AST yang dibangun pada Milestone 3, dan interpreter yang mengeksekusi instruksi P-Code tersebut. Seluruh pipeline lengkapnya adalah:

1. **Lexical Analysis** (Scanner) — tokenisasi sumber Pascal
2. **Syntax Analysis** (Parser) — pembangunan Parse Tree
3. **Semantic Analysis** (SemanticAnalyzer + SymbolTableManager) — pemeriksaan tipe dan tabel simbol
4. **Intermediate Code Generation** (CodeGenerator) — pembangkitan instruksi P-Code
5. **Execution** (Interpreter + StackMachine) — eksekusi P-Code

1.1 Struktur File Milestone 4

Berikut file-file baru dan modifikasi yang dilakukan pada milestone ini:

Path	Peran	Deskripsi Singkat
src/intermediate/CodeGenerator.hpp	Role 4	Header CodeGenerator
src/intermediate/CodeGenerator.cpp	Role 4	Visitor AST untuk emit P-Code
src/intermediate/Instruction.hpp	Role 4	Definisi kelas Instruction
src/intermediate/Instruction.cpp	Role 4	Implementasi Instruction::toString()
src/runtime/StackMachine.hpp	Role 3	Header mesin stack
src/runtime/StackMachine.cpp	Role 3	Operasi stack dan frame
src/runtime/Interpreter.hpp	Role 3	Header interpreter P-Code
src/runtime/Interpreter.cpp	Role 3	Implementasi interpreter
src/driver_m4.cpp	Role 4	Driver end-to-end M4

Tabel 1: File-file kunci Milestone 4

2 Implementasi Intermediate Code Generator

Pembangkitan kode perantara (*intermediate code generation*) adalah fase di mana *Decorated AST* dari tahap sebelumnya ditransformasi menjadi serangkaian instruksi mesin abstrak. Dalam proyek ini, kami menggunakan instruksi berbasis P-Code yang dirancang untuk dieksekusi pada mesin *stack*.

2.1 Arsitektur Instruksi P-Code

Setiap instruksi P-Code direpresentasikan oleh struktur **Instruction** yang terdiri dari empat *field* utama:

- **line**: Indeks atau alamat instruksi dalam memori instruksi.
- **f** (*function/opcode*): Kode operasi yang menentukan tindakan yang harus diambil.
- **l** (*level*): Perbedaan tingkat leksikal (*lexical level*) untuk akses variabel non-lokal.
- **a** (*argument/operand*): Nilai literal atau alamat memori.

Berikut adalah daftar lengkap instruksi P-Code yang diimplementasikan:

Opcode	Nama	Deskripsi
LIT	<i>Literal</i>	Mendorong (<i>push</i>) nilai literal a ke atas <i>stack</i> .
OPR	<i>Operator</i>	Melakukan operasi aritmatika atau logika yang ditentukan oleh a .
LOD	<i>Load</i>	Mengambil nilai dari alamat a pada tingkat leksikal l ke <i>stack</i> .
STO	<i>Store</i>	Menyimpan nilai teratas <i>stack</i> ke alamat a pada tingkat leksikal l .
LODA	<i>Load Array</i>	Mengambil nilai <i>array</i> dengan <i>offset</i> dari <i>stack</i> .
STOA	<i>Store Array</i>	Menyimpan nilai ke <i>array</i> dengan <i>offset</i> dari <i>stack</i> .
CAL	<i>Call</i>	Memanggil prosedur atau fungsi pada alamat a .
INT	<i>Increment</i>	Mengalokasikan ruang memori sebesar a pada <i>stack</i> .
JMP	<i>Jump</i>	Melompat secara tidak bersyarat ke instruksi di alamat a .
JPC	<i>Jump Conditional</i>	Melompat ke a jika nilai teratas <i>stack</i> adalah 0 (<i>false</i>).
RET	<i>Return</i>	Kembali dari pemanggilan subprogram.

Tabel 2: Instruksi P-Code yang diimplementasikan

2.2 Operasi Aritmatika dan Logika (OPR)

Instruksi OPR menggunakan argumen **a** untuk menentukan jenis operasi yang dilakukan terhadap elemen-elemen di atas *stack*.

a	Operasi	a	Operasi
1	Negasi (-)	9	Kurang dari (<)
2	Penjumlahan (+)	10	Lebih besar sama dengan (>=)
3	Pengurangan (-)	11	Lebih besar dari (>)
4	Perkalian (*)	12	Kurang dari sama dengan (<=)
5	Pembagian (/, div)	13	<i>Write</i> (Cetak nilai)
6	Modulo (mod)	14	<i>Writeln</i> (Cetak baris baru)
7	Sama dengan (=)	15	<i>Odd</i> (Ganjil)
8	Tidak sama dengan (<>)	16	Logika NOT

Tabel 3: Sub-kode operasi untuk instruksi OPR

2.3 Strategi Pembangkitan Kode

`CodeGenerator` mengimplementasikan *Visitor Pattern* untuk menelusuri *Decorated AST*. Beberapa strategi kunci yang diterapkan adalah:

1. **Pengelolaan Label dan *Backpatching*:** Untuk instruksi kontrol alur seperti IF dan WHILE, alamat lompatan (*jump target*) seringkali belum diketahui saat instruksi JMP atau JPC pertama kali dipancarkan. Kami menggunakan mekanisme label dan fungsi *backpatch* untuk mengisi alamat tersebut setelah seluruh blok kode selesai dibangkitkan.
2. **Alokasi Memori:**
 - **Variabel Lokal:** Dialokasikan mulai dari *offset* 3 (setelah *Static Link*, *Dynamic Link*, dan *Return Address*).
 - **Parameter:** Karena argumen didorong ke *stack* sebelum pemanggilan prosedur, parameter diakses menggunakan *offset* negatif relatif terhadap *base pointer* baru.
3. **Akses Array:** Pembangkitan kode untuk akses *array* melibatkan perhitungan indeks yang kemudian dikalikan dengan ukuran elemen (*element size*) untuk mendapatkan *offset* yang digunakan oleh LODA atau STOA.

3 Implementasi Interpreter dan Stack Machine

Fase eksekusi melibatkan mesin abstrak yang menjalankan instruksi P-Code yang telah dihasilkan. Komponen utamanya adalah *Stack Machine* sebagai pengelola memori dan *Interpreter* sebagai pengontrol eksekusi.

3.1 Model Memori *Stack Machine*

Stack Machine menggunakan sebuah *vector* sebagai representasi memori linier yang dikelola sebagai *stack*. Struktur *Activation Record* (Frame) untuk setiap pemanggilan prosedur terdiri dari:

1. **Static Link (SL)**: Menunjuk ke *base pointer* dari tingkat leksikal satu tingkat di atasnya untuk mendukung akses variabel non-lokal (*nested scopes*).
2. **Dynamic Link (DL)**: Menunjuk ke *base pointer* dari pemanggil (*caller*) untuk pemulihan *stack* saat kembali (*return*).
3. **Return Address (RA)**: Alamat instruksi berikutnya yang harus dieksekusi setelah prosedur selesai.
4. **Data Lokal**: Ruang untuk variabel lokal dan parameter.

Selain itu, digunakan mekanisme **Display** untuk mempercepat akses ke variabel pada tingkat leksikal yang berbeda tanpa harus menelusuri rantai *Static Link* secara berulang.

3.2 Mekanisme Eksekusi *Interpreter*

Interpreter bekerja dalam sebuah *loop* utama yang melakukan siklus *fetch-decode-execute*:

1. **Fetch**: Mengambil instruksi dari memori instruksi berdasarkan nilai *Instruction Pointer* (ip).
2. **Decode**: Mengidentifikasi *opcode* dan argumen instruksi menggunakan struktur *switch-case*.
3. **Execute**: Melakukan operasi yang sesuai, seperti memodifikasi *stack*, mengubah ip (untuk lompatan), atau memanipulasi frame aktivasi.

3.3 Penanganan Kesalahan *Runtime*

Sistem dilengkapi dengan pendeteksi kesalahan saat eksekusi (*Runtime Error handling*), meliputi:

- *Division by Zero*: Kesalahan saat operasi pembagian atau modulo dengan pembagi nol.
- *Stack Overflow/Underflow*: Melindungi integritas memori dari penggunaan berlebihan atau pengambilan data dari *stack* kosong.
- *Index Out of Bounds*: Memastikan akses *array* berada dalam rentang yang dideklarasikan.
- *Invalid Jump*: Mencegah lompatan ke alamat instruksi yang tidak valid.

4 Pengujian

Berikut hasil pengujian untuk lima kasus uji wajib. Semua pengujian berhasil menghasilkan output yang sesuai.

4.1 Kasus 1: Program Dasar (pos_basic.txt)

Program paling sederhana: inisialisasi variabel, lalu cetak nilainya.

```
1 program Basic;  
2 var x: integer;  
3 begin  
4     x := 42;  
5     writeln(x);  
6 end.
```

Output:

```
1 42
```

4.2 Kasus 2: Aritmatika (pos_arith.txt)

Menguji urutan operasi: perkalian lebih tinggi prioritas dari penjumlahan.

```
1 program Arith;  
2 var x: integer;  
3 begin  
4     x := 10 + 20 * 3;  
5     writeln(x);  
6 end.
```

Output:

```
1 70
```

4.3 Kasus 3: If-Else (pos_if_else.txt)

Percabangan dengan kondisi >.

```
1 program IfElse;  
2 var x, y: integer;  
3 begin  
4     x := 10;  
5     if x > 5 then  
6         y := 1  
7     else  
8         y := 0;  
9     writeln(y);  
10 end.
```

Output:

```
1 1
```

4.4 Kasus 4: While Loop (pos_while.txt)

Perulangan while dengan pencetakan nilai menurun.

```
1 program WhileLoop;  
2 var i: integer;  
3 begin  
4     i := 5;  
5     while i > 0 do  
6         begin  
7             writeln(i);  
8             i := i - 1;  
9         end;  
10 end.
```

Output:

```
1 5  
2 4  
3 3  
4 2  
5 1
```

4.5 Kasus 5: For Loop (pos_for.txt)

Perulangan for dengan pencetakan nilai 1 sampai 5.

```
1 program ForLoop;  
2 var i: integer;  
3 begin  
4     for i := 1 to 5 do  
5         begin  
6             writeln(i);  
7         end;  
8 end.
```

Output:

```
1 1  
2 2  
3 3  
4 4  
5 5
```

4.6 Kasus Tambahan: Prosedur dengan Parameter

Menguji pemanggilan prosedur dengan satu parameter integer.

```
1 program ProcParam;  
2  
3 var x: integer;  
4  
5 procedure printNum(n: integer);
```



```
6  begin
7      writeln(n);
8  end;
9
10 begin
11     x := 42;
12     printNum(x);
13 end.
```

Output:

```
1  42
```

5 Kesimpulan

Milestone 4 berhasil mengimplementasikan seluruh rantai kompilasi dari analisis leksikal hingga eksekusi kode perantara. Pemecahan masalah parameter passing menjadi hasil penting: dengan menggunakan alamat negatif untuk parameter (karena argumen sudah berada di stack sebelum `CAL`) dan memancarkan `INT` hanya untuk variabel lokal, pemanggilan prosedur dan fungsi dapat berjalan dengan benar. Semua lima kasus uji wajib serta kasus tambahan berhasil dilalui tanpa regresi pada fitur-fitur sebelumnya.